



计算机视觉-Opencv入门

教师： 王一帆
助教： 张文博

虚拟环境

日常使用中，我们的python版本和各种库版本不相同，针对不同的项目要使用不同的版本，比如今天我做项目A要求python版本为3.7，明天我项目B要求python版本为3.8，那么我们如何管理不同的版本呢？

虚拟环境

conda env list

```
(base) C:\Users\zwb12>conda env list
# conda environments:
#
base                *  E:\Anaconda
cv2                  E:\Anaconda\envs\cv2
```

新建虚拟环境

```
conda create -n env_name python=3.7
```

The following NEW packages will be INSTALLED:

ca-certificates	pkgs/main/win-64::ca-certificates-2023.08.22-haa95532_0
certifi	pkgs/main/win-64::certifi-2022.12.7-py37haa95532_0
openssl	pkgs/main/win-64::openssl-1.1.1w-h2bbff1b_0
pip	pkgs/main/win-64::pip-22.3.1-py37haa95532_0
python	pkgs/main/win-64::python-3.7.16-h6244533_0
setuptools	pkgs/main/win-64::setuptools-65.6.3-py37haa95532_0
sqlite	pkgs/main/win-64::sqlite-3.41.2-h2bbff1b_0
vc	pkgs/main/win-64::vc-14.2-h21ff451_1
vs2015_runtime	pkgs/main/win-64::vs2015_runtime-14.27.29016-h5e58377_2
wheel	pkgs/main/win-64::wheel-0.38.4-py37haa95532_0
wincertstore	pkgs/main/win-64::wincertstore-0.2-py37haa95532_2

Proceed ([y]/n)? |

新建虚拟环境

输入y确认

```
Downloading and Extracting Packages

Preparing transaction: done
Verifying transaction: done
Executing transaction: done
#
# To activate this environment, use
#
#     $ conda activate test
#
# To deactivate an active environment, use
#
#     $ conda deactivate

(base) C:\Users\zwb12>|
```

激活虚拟环境

conda activate env_name

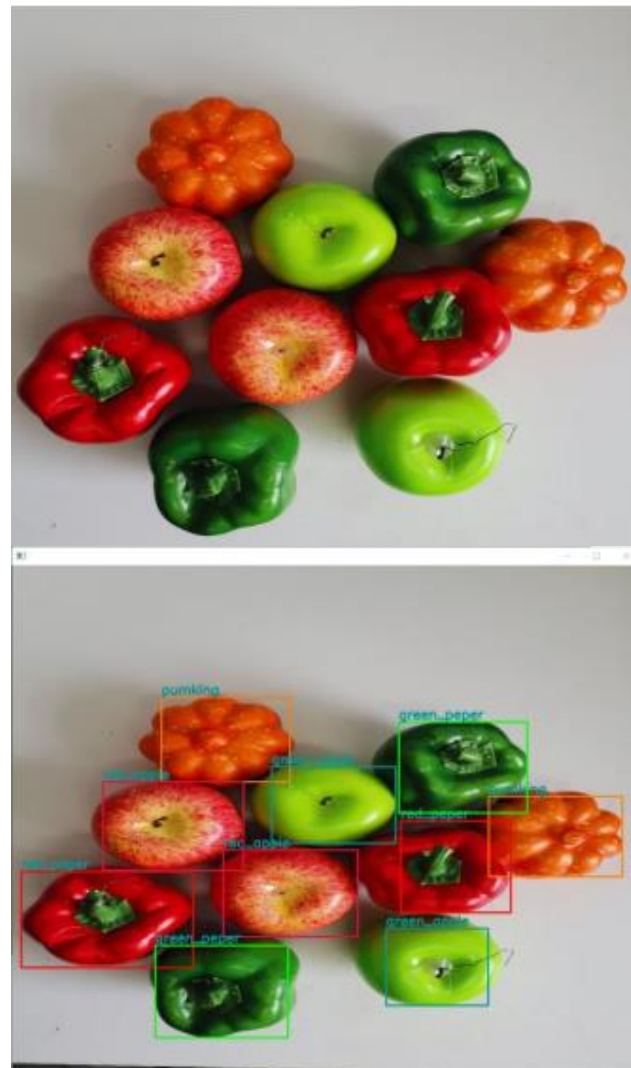
```
(base) C:\Users\zwb12>conda activate test  
(test) C:\Users\zwb12>|
```

退出虚拟环境

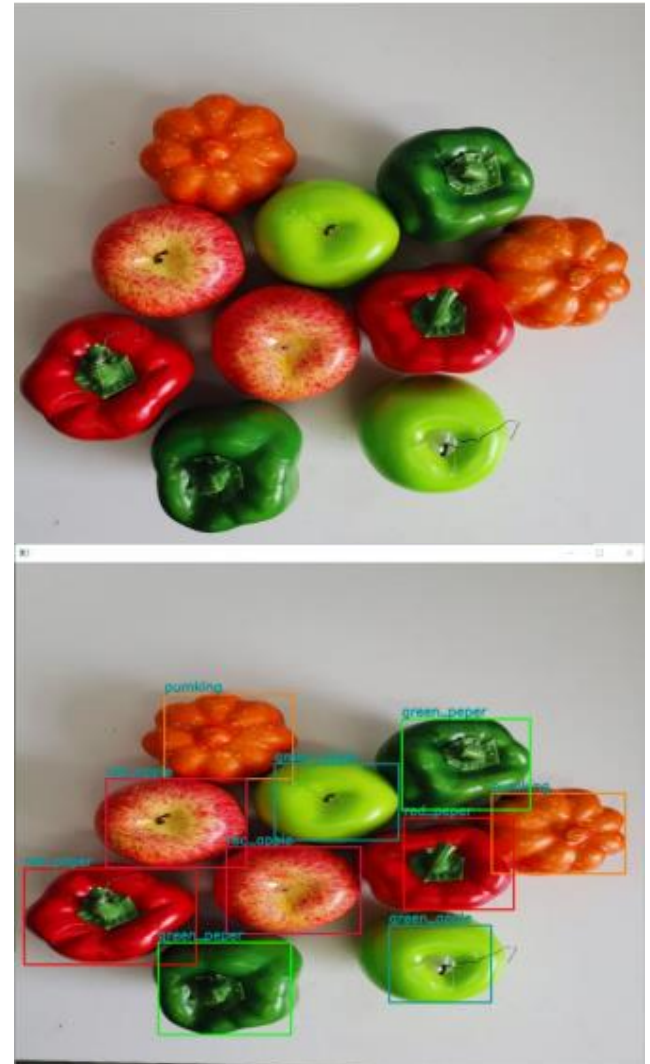
conda deactivate

```
(test) C:\Users\zwb12>conda deactivate  
(base) C:\Users\zwb12>|
```

- 利用OpenCV完成对给定图像的操作
- 要求：
 - ① 能够打开一幅图片，并显示；
 - ② 实现图像中特定目标的提取；



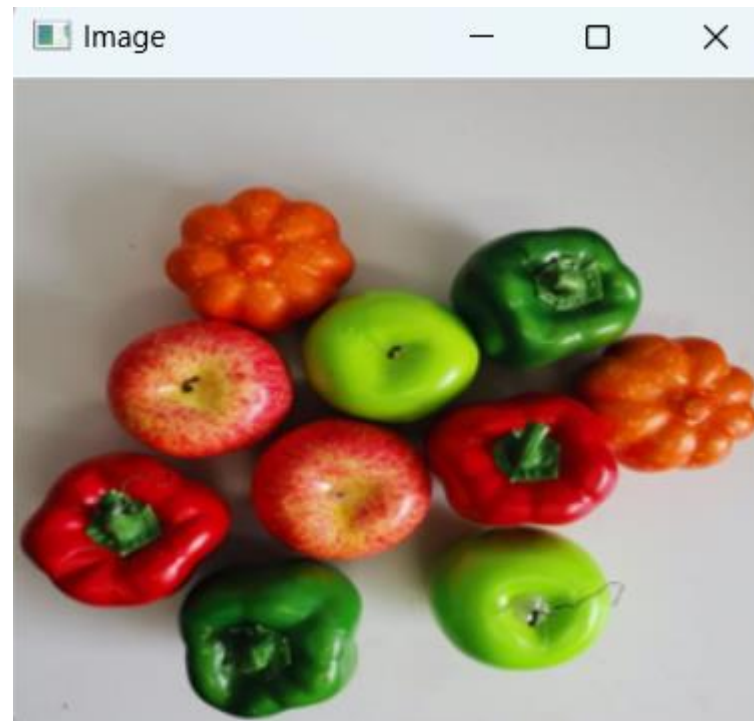
- 利用OpenCV完成对给定图像的操作
- 要求：
 - 能够打开一幅图片，并显示；
 - 读取图像
 - 显示图片
 - 实现图像中特定目标的提取；
 - 获取轮廓
 - 根据轮廓绘制方框



打开一幅图片，并显示

- ① 导入所需库
- ② 用cv2.imread()读取图片
- ③ cv2.imshow()显示图片

```
def display(image_path):  
    # 使用OpenCV打开指定路径的图像并显示  
    image = cv2.imread(image_path)  
    cv2.imshow('Image', image)  
    return image
```



实现图像中特定目标的提取

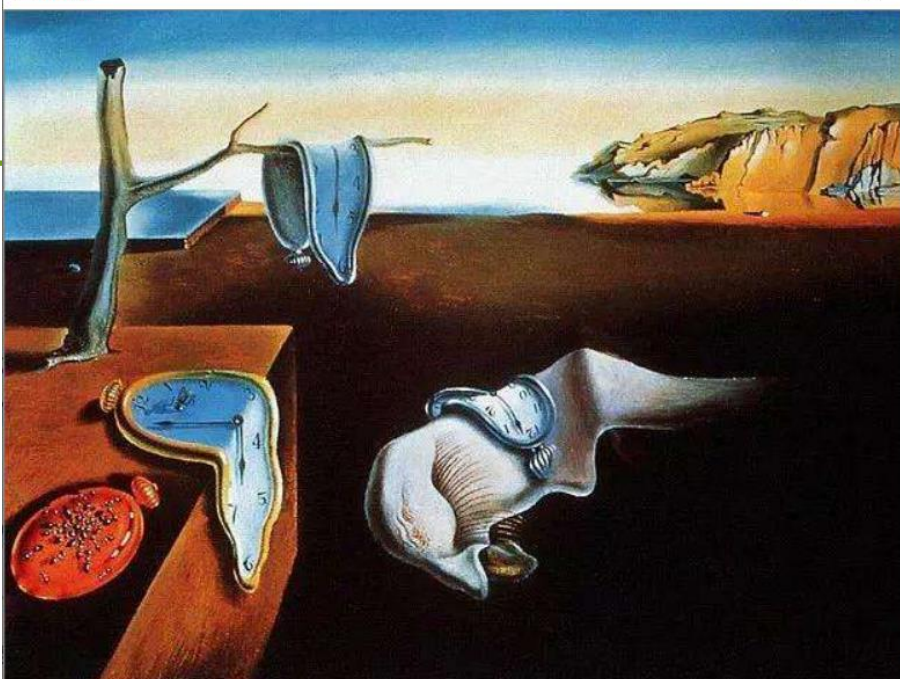
- ① 颜色差异
- ② 形状大小
- ③ 绘制矩形和文字



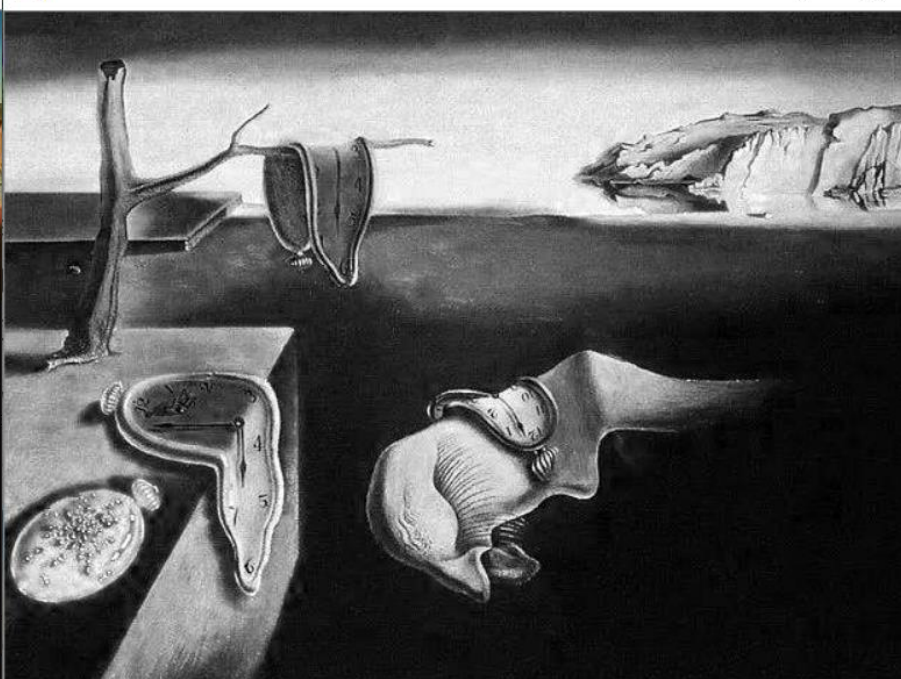
- 颜色模型是一个使用原色构建全系列颜色的系统。原色由红色、绿色和蓝色(RGB)组成。
- `opencv`中用不同的矩阵来存放各个颜色通道中的数据，存放顺序是（BGR）

```
list=['b' , 'g' , 'r']  
cv2.imshow('origin',img)  
for i in range(3):  
    cv2.imshow(list[i],img[:, :, i])  
cv2.waitKey(0)  
cv2.destroyAllWindows()
```

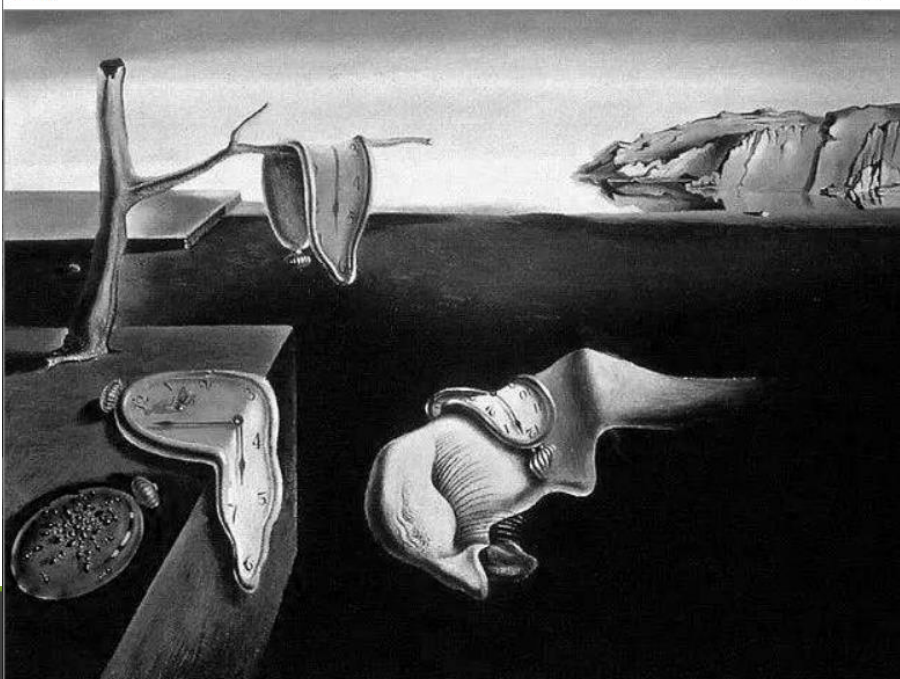
origin



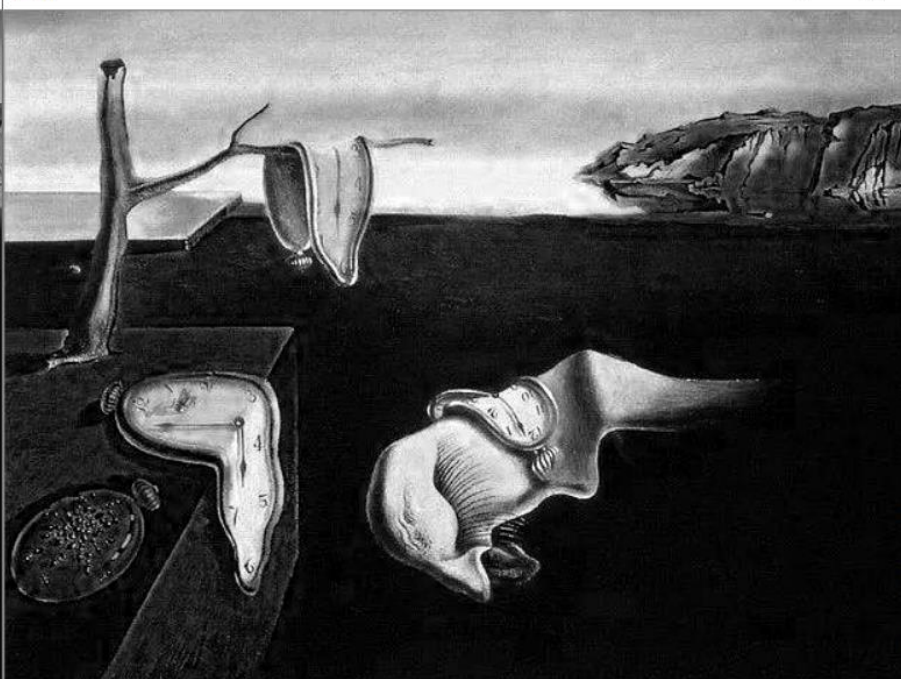
r



g



b



- RGB三色都为0的时候，表示黑色。
- RGB三色都为255的时候，表示白色。
- 当0的时候，能量越低，故为黑色。数值变大，则能量越来越大，则越来越往白色靠。

RGB各通道显示

```
1 import cv2
2 img_path = 'test.jpg'
3 img = cv2.imread(img_path)
4 print(img)
```

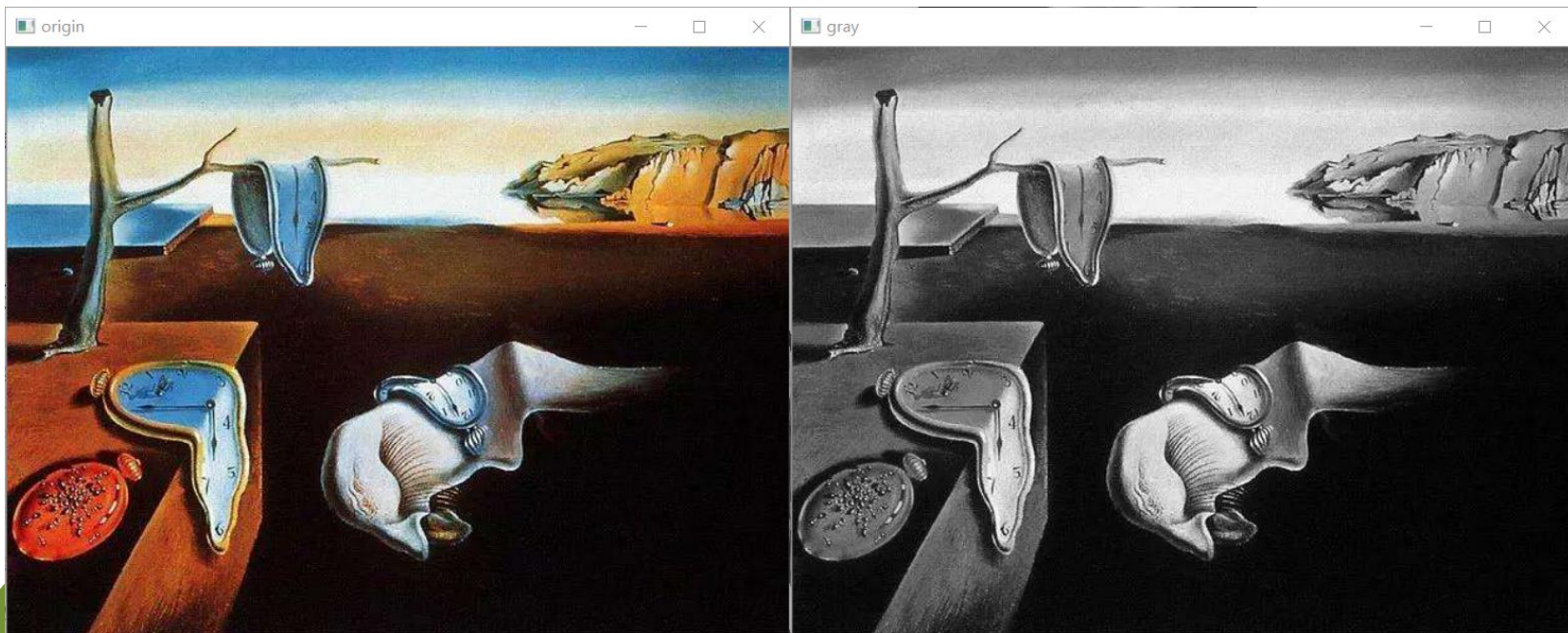


```
[[[179 162 136]
   [247 230 204]
   [248 231 205]
   ...
   [229 177 136]
   [221 170 130]
   [205 154 114]]

  [[179 162 136]
   [247 230 204]
   [248 231 205]
   ...
   [229 177 136]
   [221 170 130]
   [205 154 114]]

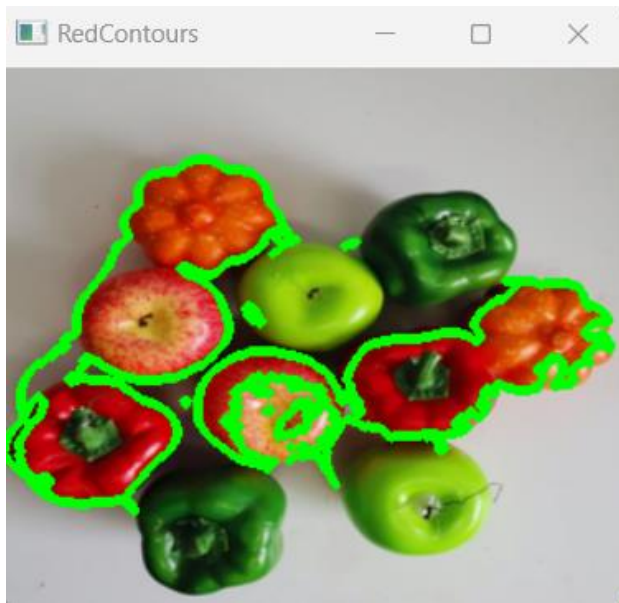
  [[179 162 136]
   [247 230 204]
   [248 231 205]
   ...
```

- 灰度图，Gray Scale Image 或是 Grey Scale Image，又称**灰阶图**。把白色与黑色之间按对数关系分为若干等级，称为灰度。灰度分为256阶。
- 用灰度表示的图像称作灰度图。在图像处理中常将RGB图像转为灰度图再做处理。



通过颜色阈值筛选目标

- ① 设置红色和绿色的阈值上下限
- ② 利用`cv2.inRang()`根据阈值分别得到红色水果和绿色水果的灰度图
- ③ 利用`cv2.findContours()`处理灰度图，分别得到红色和绿色水果轮廓



通过颜色阈值筛选目标

① 设置红色和绿色的阈值上下限



通过颜色阈值筛选目标

① 设置红色和绿色的阈值上下限

```
15 def detect(img_path):  
16     img = cv2.imread(image_path)  
17     # 定义红色和绿色范围的下限和上限  
18     lower_red = np.array([0, 0, 100])  
19     upper_red = np.array([100, 100, 255])  
20  
21     lower_green = np.array([0, 100, 0])  
22     upper_green = np.array([100, 255, 100])
```

通过颜色阈值筛选目标

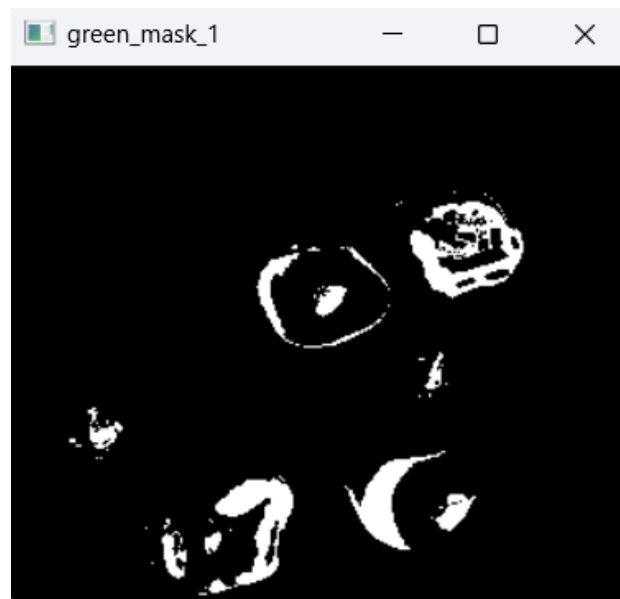
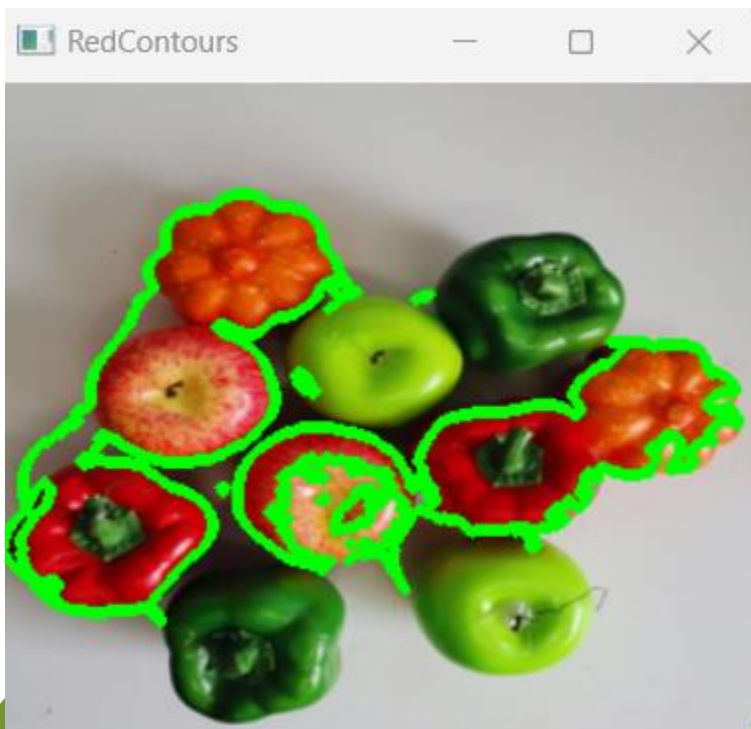
② 根据阈值分别得到红色水果和绿色水果的灰度图

用cv2.inRange()取范围

```
24      # 使用颜色范围进行红色和绿色水果的检测
25      red_mask = cv2.inRange(img, lower_red, upper_red)
26      cv2.imshow('red_mask', red_mask)
27      green_mask = cv2.inRange(img, lower_green, upper_green)
28      cv2.imshow('green_mask', green_mask)
```

通过颜色阈值筛选目标

`cv2.inRange()`通过对比原图和上下界值，返回一个二值化的掩码图像
我们通过`cv2.imshow()`展示出来



通过颜色阈值筛选目标

③ 利用cv2.findContours()处理灰度图，分别得到红色和绿色水果轮廓

我们以红色水果为例

```
red_contours, _ = cv2.findContours(red_mask,  
cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_NONE)
```

cv2.findContours()

图像轮廓是指图像中物体边缘的连续性曲线。在计算机视觉和图像处理中，轮廓通常被用于检测物体、分割图像以及提取物体特征。

图像轮廓是由一系列连续的像素点组成，这些像素点位于物体边界上。轮廓的特点是在物体和背景之间的边界位置，因此可以用来表示物体的形状和结构。轮廓可以是闭合的，也可以是开放的，具体取决于物体的形状。

cv2.findContours()

`contours, hierarchy = cv2.findContours(image, mode, method)`

返回值：

`contours`：返回的轮廓。

`hierarchy`：图像的拓扑信息（轮廓层次）。

参数：

`image`：原始图像。

`mode`：轮廓检索模式。

`method`：轮廓的近似方法。

```
contours, hierarchy = cv2.findContours(image, mode, method)
```

参数详解：

image

原始图像。8 位单通道图像，所有非零值被处理为 1，所有零值保持不变。也就是说灰度图像会被自动处理为二值图像。在实际操作时，可以根据需要，预先使用阈值处理等函数将待查找轮廓的图像处理为二值图像。

`contours, hierarchy = cv2.findContours(image, mode, method)`

参数详解：

mode

参数 mode 决定了轮廓的提取方式，具体有如下几种：

`cv2.RETR_EXTERNAL`：只检测外轮廓。

另外还有 `cv2.RETR_LIST`，`cv2.RETR_CCOMP`，`cv2.RETR_TREE`

`contours, hierarchy = cv2.findContours(image, mode, method)`

参数详解：

method

•参数 method 决定了如何表达轮廓，常用为以下两种值：

cv2.CHAIN_APPROX_NONE：存储所有的轮廓点，相邻两个点的像素位置差不超过 1，即 $\max(\text{abs}(x1-x2), \text{abs}(y2-y1))=1$ 。

•cv2.CHAIN_APPROX_SIMPLE：压缩水平方向、垂直方向、对角线方向的元素，只保留该方向的终点坐标。例如，在极端的情况下，一个矩形只需要用 4 个点来保存轮廓信息。

`contours, hierarchy = cv2.findContours(image, mode, method)`



图 12-6 使用不同的 `method` 参数值所保存的轮廓点

CSDN @玩转AI

`contours, hierarchy = cv2.findContours(image, mode, method)`

参数详解：

返回值 contours

该返回值返回的是一组轮廓信息，每个轮廓都是由若干个点所构成的。
例如，`contours[i]` 是第 i 个轮廓（下标从 0 开始），`contours[i][j]` 是第 i 个轮廓内的第 j 个点。

`contours, hierarchy = cv2.findContours(image, mode, method)`

图 12-1 所示为提取的轮廓示例，函数 `cv2.findContours()` 提取出左图的 3 个轮廓，每一个轮廓都是由若干个像素点构成的。

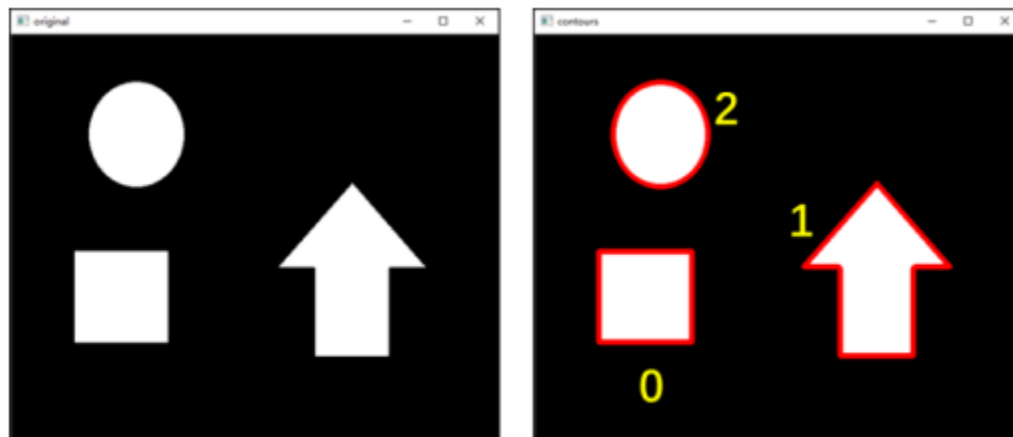


图 12-1 轮廓示例

CSDN @玩转AI

```
contours, hierarchy = cv2.findContours(image, mode, method)
```

返回值contours类型是一个list，list 的每个元素都是图像的一个轮廓，用 Numpy中的 ndarray 结构表示。
每一个轮廓都是由若干个像素点构成的，点的个数不固定，具体个数取决于轮廓的形状。

```
print (len(contours[0])) #打印第 0 个轮廓的长度 (点的个数) : 4  
print (len(contours[1])) #打印第 1 个轮廓的长度 (点的个数) : 60  
print (len(contours[2])) #打印第 2 个轮廓的长度 (点的个数) : 184  
print (contours[0]) #打印第 0 个轮廓中的像素点
```

```
[[[ 79 270]]  
 [[ 79 383]]  
 [[195 383]]  
 [[195 270]]]
```

`contours, hierarchy = cv2.findContours(image, mode, method)`

参数详解：
返回值hierarchy

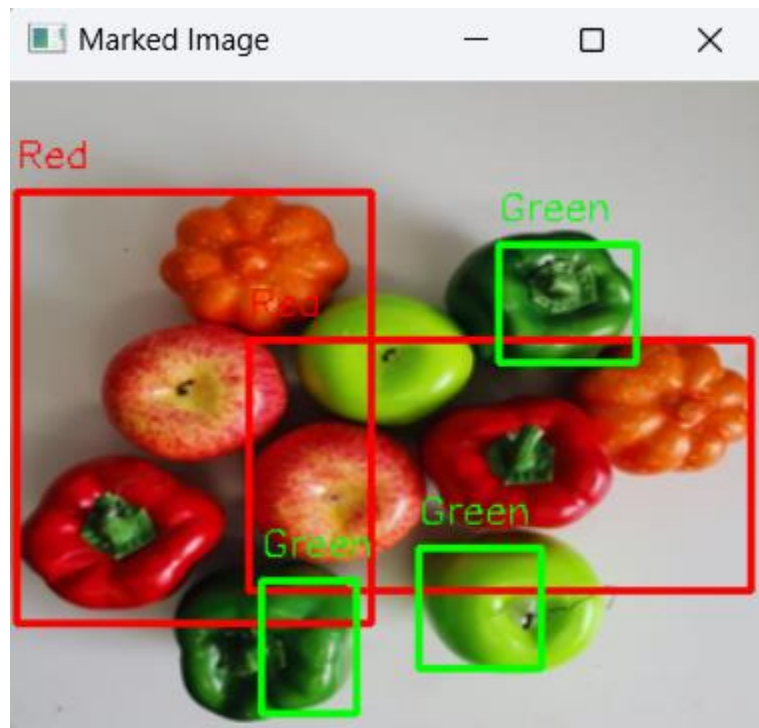
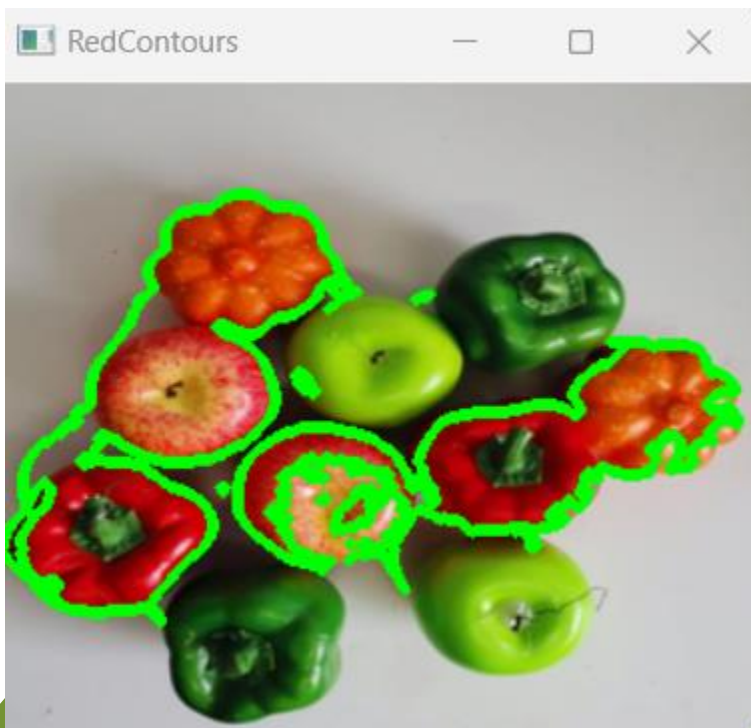
图像内的轮廓可能位于不同的位置。比如，一个轮廓在另一个轮廓的内部。

详细可见：



处理轮廓

- ① 通过cv2.contourArea()计算轮廓面积，过滤较小面积的轮廓
- ② 利用cv2.boundingRect()框住轮廓
- ③ 根据方框坐标绘制提取结果



① 通过cv2.contourArea()计算轮廓面积，过滤较小面积的轮廓

cv2.contourArea函数接受一个轮廓作为参数，返回该轮廓的面积。

area = cv2.contourArea(contour)

```
area = cv2.contourArea(contour)
if area > 300: # 设置面积阈值，可根据实际情况调整
```



② 利用cv2.boundingRect()框住轮廓

cv2.contourArea函数能够计算给定轮廓的面积。在使用该函数之前，需要先使用cv2.findContours函数找到图像中的轮廓。

cv2.contourArea函数接受一个轮廓作为参数，返回该轮廓的面积。

`x, y, w, h = cv2.boundingRect(contour)`

③ 根据方框坐标绘制提取结果

`cv2.rectangle(img, pt1, pt2, color, thickness, lineType)`

`cv2.rectangle(img, (x, y), (x + w, y + h), (0, 0, 255), 2)`

处理轮廓

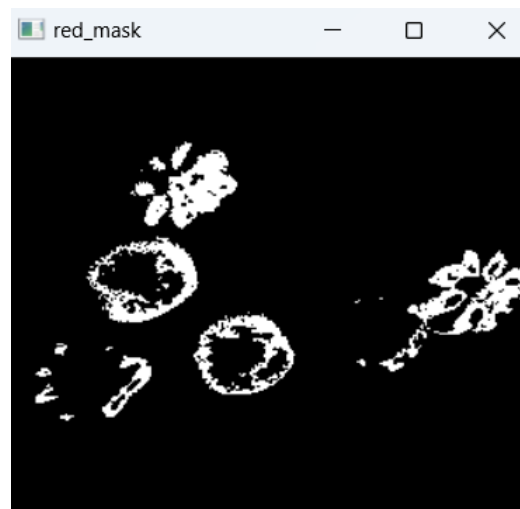
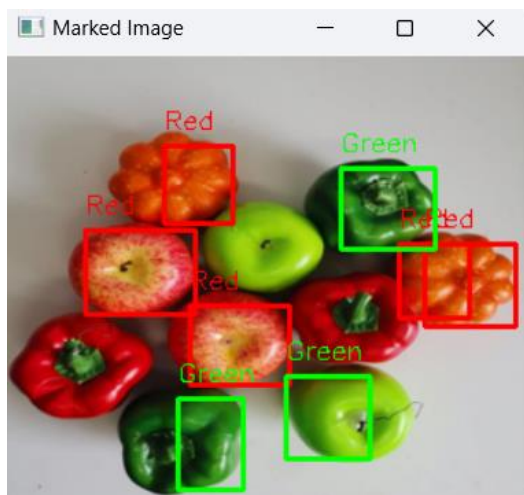
绿色同理

```
# 找到绿色水果的轮廓并标记
green_contours, _ = cv2.findContours(green_mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_NONE)
for contour in green_contours:
    area = cv2.contourArea(contour)
    if area > 500: # 设置面积阈值, 可根据实际情况调整
        x, y, w, h = cv2.boundingRect(contour)
        cv2.rectangle(img, (x, y), (x + w, y + h), (0, 255, 0), 2)
        cv2.putText(img, 'Green', (x, y - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 1)
```

轮廓连接在一起

调整颜色阈值范围

```
# 定义红色和绿色范围的下限和上限  
lower_red = np.array([0, 0, 200])  
upper_red = np.array([100, 120, 255])
```



难以找到合适阈值如何处理？

多组阈值，多组轮廓

```
# 定义红色和绿色范围的下限和上限
```

```
lower_red_1 = np.array([0, 0, 140])  
upper_red_1 = np.array([30, 30, 255])  
lower_red_2 = np.array([30, 30, 100])  
upper_red_2 = np.array([65, 190, 230])  
lower_red_3 = np.array([0, 40, 150])  
upper_red_3 = np.array([15, 105, 230])
```

```
lower_green_1 = np.array([0, 40, 0])  
upper_green_1 = np.array([34, 150, 56])  
lower_green_2 = np.array([7, 100, 85])  
upper_green_2 = np.array([78, 229, 191])
```

难以找到合适阈值如何处理？

多组阈值，多组轮廓

